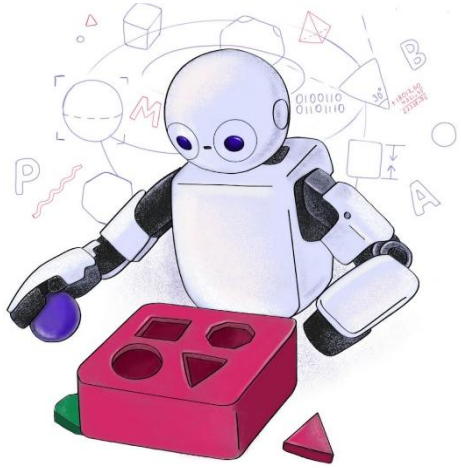
TP558 - Tópicos avançados em Machine Learning:

FANN-on-MCU: An Open-Source Toolkit for Energy-Efficient Neural Network Inference at the Edge of the Internet of Things



Inatel

Gabriel Dias Scarpioni
gabriel.scarpioni@inatel.br

Abertura

- Expansão das aplicações de Internet das Coisas (IoT):
  - Dispositivos de baixa potência.
- Necessidade de processamento de dados próximo à fonte:
  - Edge Computing.
- Limitações de hardware:
  - memória reduzida;
  - capacidade computacional limitada;
  - restrições energéticas.

Abertura

- FANN-on-MCU:
  - Toolkit open-source;
  - Fast Artificial Neural Network (FANN);
  - Geração de código otimizado para microcotroladores.
- Análise de eficiência energética entre diferentes núcleos;
- Análise das acelerações paralelas e das degradações devido à sobrecarga de paralelização e transferências de memória.
- Resultados experimentais.

Introdução

- ML aplicado em sistemas de IoT e dispositivos móveis → extrair informações de grandes volumes de dados sensoriais.
- Redes neurais apresentam alta precisão em tarefas de classificação.
- Limitação do modelo tradicional:
  - Atualmente, o processamento de ML ocorre principalmente na nuvem;
  - Dispositivos IoT enviam dados brutos ou pouco processados para processamento remoto.

Introdução

- Problema do modelo tradicional no contexto de IoT:
  - Sobrecarga da rede;
  - Maior consumo de energia na transmissão;
  - Latência;
  - Questões de privacidade e confiabilidade.

Introdução

- Edge Computing como alternativa.
- Desafios:
  - Abordagens em computação em nuvem utiliza redes neurais convolucionais profundas;
  - Energia (alimentação por bateria);
  - Memória;
  - Capacidade de processamento.

Introdução

- Objetivo: Desenvolver um framework que permita executar redes neurais em microcontroladores de baixo consumo de forma eficiente.

- É possível executar redes neurais eficientes em microcontroladores de ultra-baixo consumo, desde que:

  - os algoritmos sejam leves (MLP);

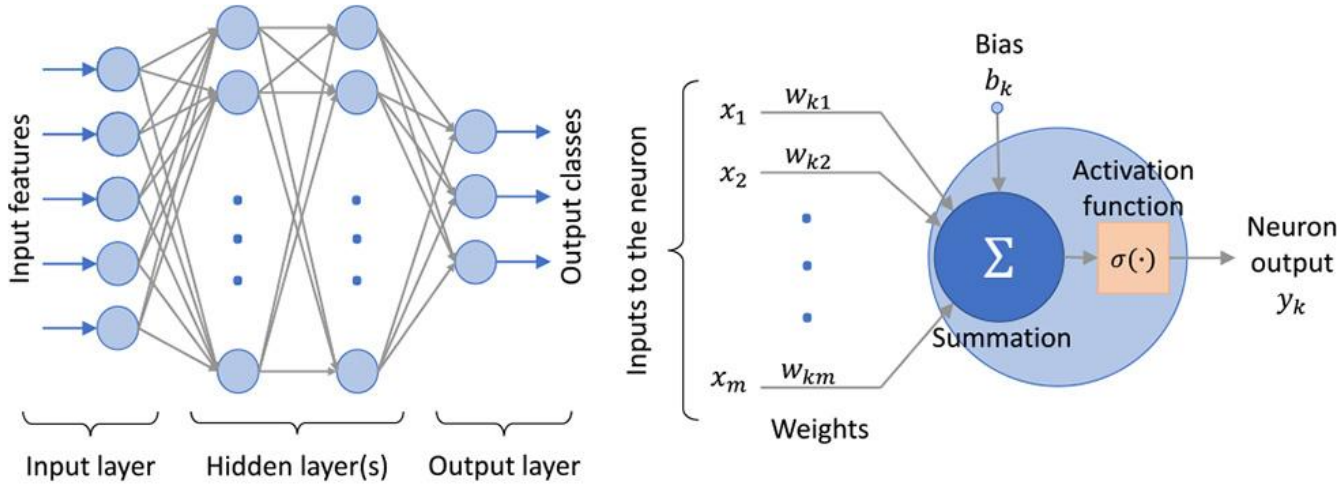
  - o software seja otimizado para a arquitetura do hardware;

  - sejam exploradas técnicas de paralelismo e otimização de memória.

Introdução

- Contribuições:
  - FANN-on-MCU para processadores ARM e RISC-V;
  - Comparações de desempenho e uso de memória;
  - Otimizações para diferentes tamanhos de rede e hierarquias de memória;
  - Análise detalhada de ganhos e perdas por paralelização;
  - Implementação em cenários reais, como reconhecimento de gestos, detecção de quedas e classificação de atividades humanas;
  - Resultados experimentais com medições de campo;
  - Lançamento do kit de ferramentas como software de código aberto.

Fundamentação teórica - Multilayer Perceptron



$$x_k^{(\ell+1)}(\mathbf{x}) = \sigma\left(\sum_{i=1}^{m} w_{k,i}^{(\ell)} x_i^{(\ell)} + b_k\right)$$

Fundamentação teórica – FANN

- Framework leve para redes neurais MLP, voltado para treinamento e inferência de redes neurais artificiais.
- Implementado em linguagem C e suporte a múltiplas linguagens
- Ferramentas de ajuste automático de hiperparâmetros, auxiliando na escolha da melhor configuração da rede neural.
- Suporte a inferência em ponto fixo (fixed-point), permitindo execução eficiente em microcontroladores de baixo consumo.
- Treinamento em cascata (cascade training), que adiciona neurônios automaticamente durante o treinamento para otimizar a arquitetura da rede.
- Formato simples de arquivos para armazenamento de datasets e modelos de redes neurais treinadas.
- Interface gráfica FANNTool

Fundamentação teórica – Frameworks

- STM32Cube.AI
  - Ferramenta da STMicroelectronics para implantação de redes neurais em microcontroladores STM32.
  - Permite importar modelos treinados em frameworks como Keras e TensorFlow Lite.
  - Geração automática de código C otimizado para execução em MCUs da série STM32.
  - Suporte a modelos quantizados, reduzindo o tamanho do modelo e acelerando a inferência.
  - Facilita a integração de IA embarcada em sistemas de baixo consumo energético.
- TensorFlow Lite for Microcontrollers
  - Framework independente de fabricante, suportando plataformas ARM Cortex-M e RISC-V.
  - Permite exportar modelos treinados no TensorFlow para execução em microcontroladores.
  - Runtime leve, ocupando cerca de 16 kB de memória em um Cortex-M3.

Fundamentação teórica

- CMSIS-NN
  - Implementações altamente otimizadas de camadas comuns em redes neurais
  - Acelerações de até 4,6×
- FANNCortexM
  - Treinar uma rede MLP utilizando a biblioteca Fast Artificial Neural Network Library (FANN)
  - Exportar automaticamente código executável para microcontroladores ARM Cortex-M

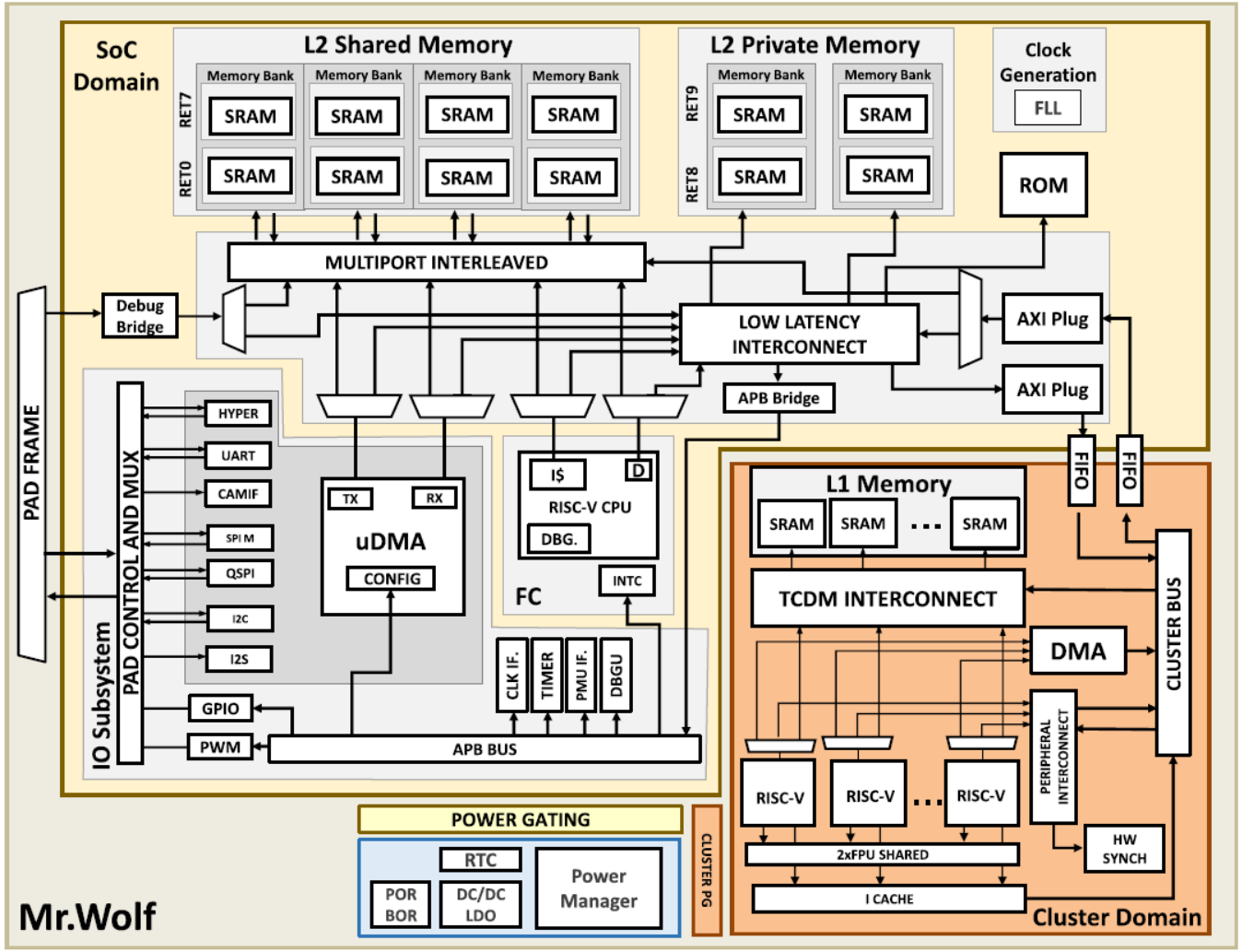
Fundamentação teórica – ARM Cortex-M Family

- ARM Cortex-M (M0, M3, M4 e M7)
- Frequência de operação: 16MHz (M0) – 300 MHz (M7)
- SRAM → 256–512 Kb
- Flash → 1 – 2 MB
- Maioria não possui unidade de ponto flutuante
- Instruções de DSP integradas → M3, M4 e M7
- Single Instruction Multiple Data (SIMD) e Multiply – and – Accumulate (MAC) → M4 e M7
- CMSIS: arm_dot_prod, arm_fill e arm_copy → até 30% melhoria

Fundamentação teórica – PULP Processors

- Parallel Ultra Low Power (PULP)
- Plataforma multicore open-source baseada em RISC-V
  - Fabric Controller + Cluster múltiplos cores + memória hierárquica + DMA
- Projetada para alta eficiência energética
- Processador Mr. Wolf

Fundamentação teórica – PULP Processors



Fundamentação teórica – PULP Processors

```
for (i = 0; i < 100; i++) {
    d[i] = a[i] + b[i];
}
```

11 cycles/output

```
mv     x5, 0
mv     x4, 100
Lstart:
  lb   x2, 0(x10)
  lb   x3, 0(x11)
  addi x10, x10, 1
  addi x11, x11, 1
  add  x2, x3, x2
  sb   x2, 0(x12)
  addi x4, x4, -1
  addi x12, x12, 1
bne    x4, x5, Lstart
```

Baseline

8 cycles/output

```
mv     x5, 0
mv     x4, 100
Lstart:
  lb   x2, 0(x10!)
  lb   x3, 0(x11!)
  addi x4, x4, -1
  add  x2, x3, x2
  sb   x2, 0(x12!)
bne    x4, x5, Lstart
```

Post-incr. load and store

5 cycles/output

```
lp.setupi 100, Lend
  lb   x2, 0(x10!)
  lb   x3, 0(x11!)
  add  x2, x3, x2
Lend: sb x2, 0(x12!)
```

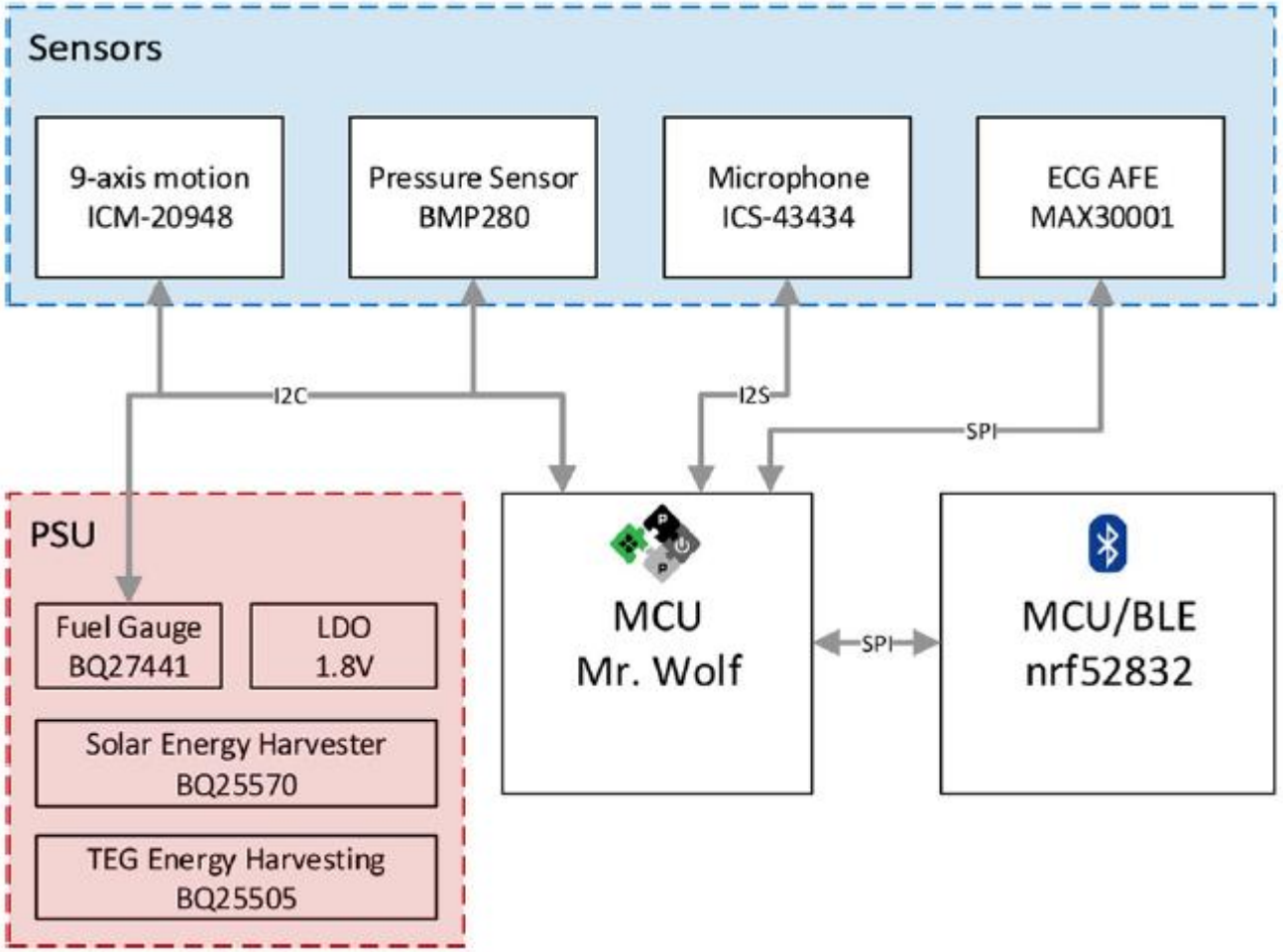
Hardware loop

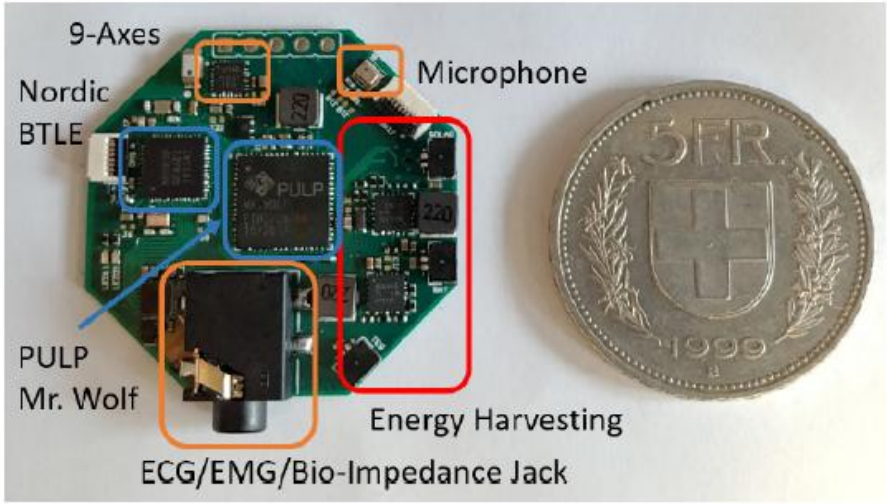
1.25 cycles/output

```
lp.setupi 25, Lend
  lw   x2, 0(x10!)
  lw   x3, 0(x11!)
  pv.add.b x2, x3, x2
Lend: sw x2, 0(x12!)
```

Packed SIMD

Fundamentação teórica – InfiniWolf





Fundamentação teórica

TABLE I
ASSEMBLY CODE FOR INNER-LOOP DOT-PRODUCT. THE NUMBER OF CYCLES TAKEN BY THE INSTRUCTION IS REPORTED IN PARENTHESIS

ARM Cortex-M4 Float	ARM Cortex-M4 Fixed	RISC-V RI5CY Float	RISC-V RI5CY Fixed

vldmia.32 (post-incr. load, 1)
vldmia.32
subs (counter subtract, 1)
vfma.f32 (fused mult-add, 3)
bne (taken conditional branch, 2)
	ldr (post-incr. load, 1)
	ldr (post-incr. load, 1)
	mul (multiply, 1)
	add (addition, 1)
	subs (counter subtract, 1)
	bne (taken conditional branch, 2)
		flw (float load, 1)
		flw
		addi (pointer incr., 1)
		addi
		fmadd.s (fused mult-add, 1)
			p.lw (post-incr. load, 1)
			p.lw
			mul (multiply, 1)
			sra (shift right arith., 1)
			add (addition, 1)

	4× loopunrolling		2× loopunrolling

FANN-on-MCU

- Script de conversão
  - Ferramenta que lê a rede neural treinada usando a biblioteca Fast Artificial Neural Network Library.
  - Converte o modelo treinado em arquivos de código C.
  - Analisa características da rede como:
    - número de neurônios
    - número de pesos
    - número de camadas
  - Pode converter a rede para ponto fixo ou ponto flutuante.

- Rotinas de execução da rede neural adaptadas para microcontroladores
  - fann_run()

Metodologia

- Análise do código do FANNCortexM → Identificar gargalos de desempenho
- Implementação do FANN-on-MCU
- Análise de desempenho considerando uma rede fixa:
  - Cinco entradas
  - Duas camadas ocultas com 100 neurônios cada
  - Três classes de saída
  - Cada neurônio seguido função tangente hiperbólica
- Realizado no Cortex M4 e no Mr. Wolf considerando ponto flutuante e fixo

Metodologia

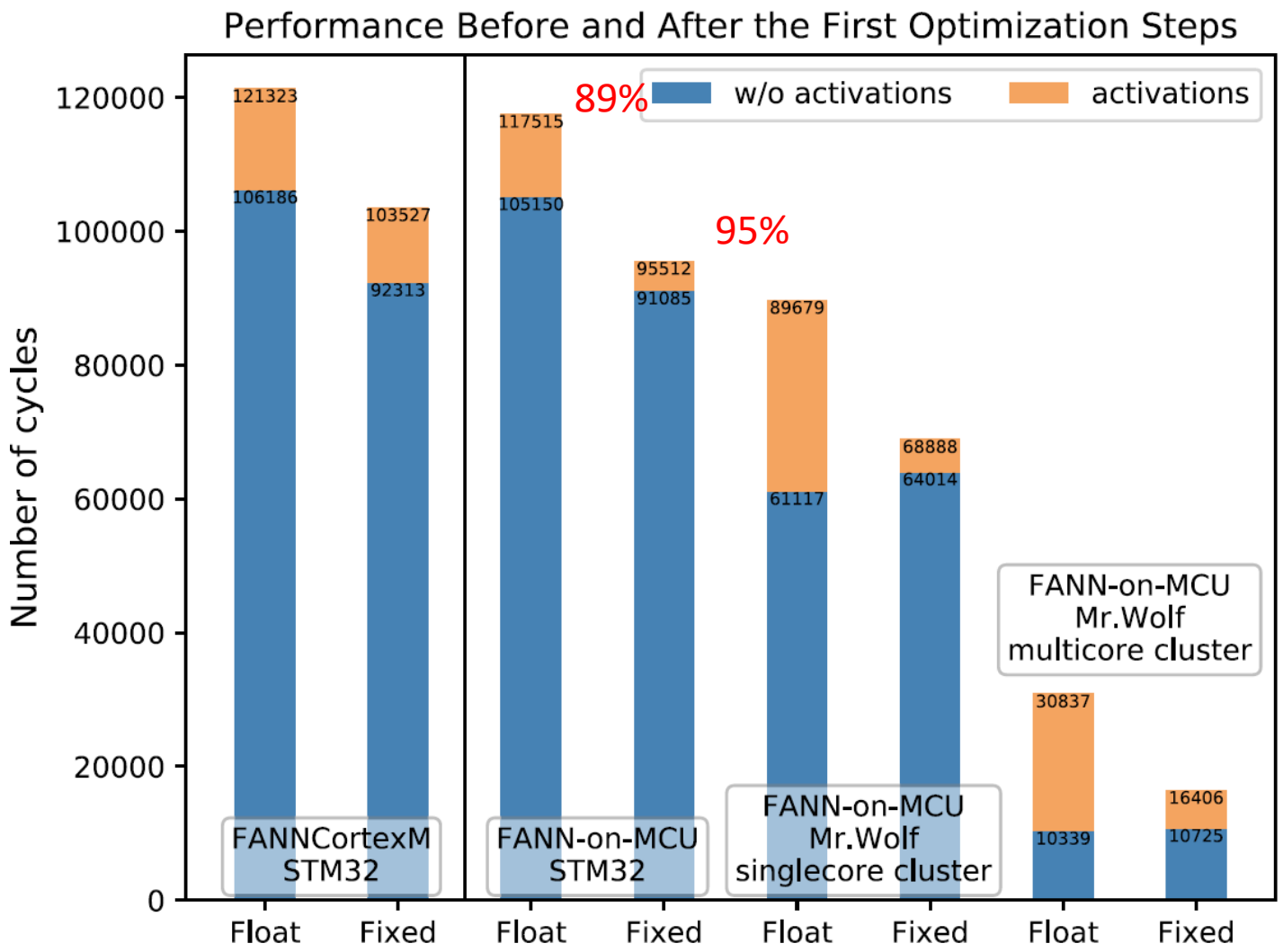
- Análise de desempenho considerando uma única camada e alterando o número de entradas e saídas

- Análise de desempenho da rede completa considerando:
  - 100 entradas e 8 saídas
  - Alterando o número de camadas ocultas
  - O número de neurônio em cada camada varia conforme:

$$N_l = (l \ \mathbf{mod} \ 2 + l \ \mathbf{div} \ 2) \cdot d$$

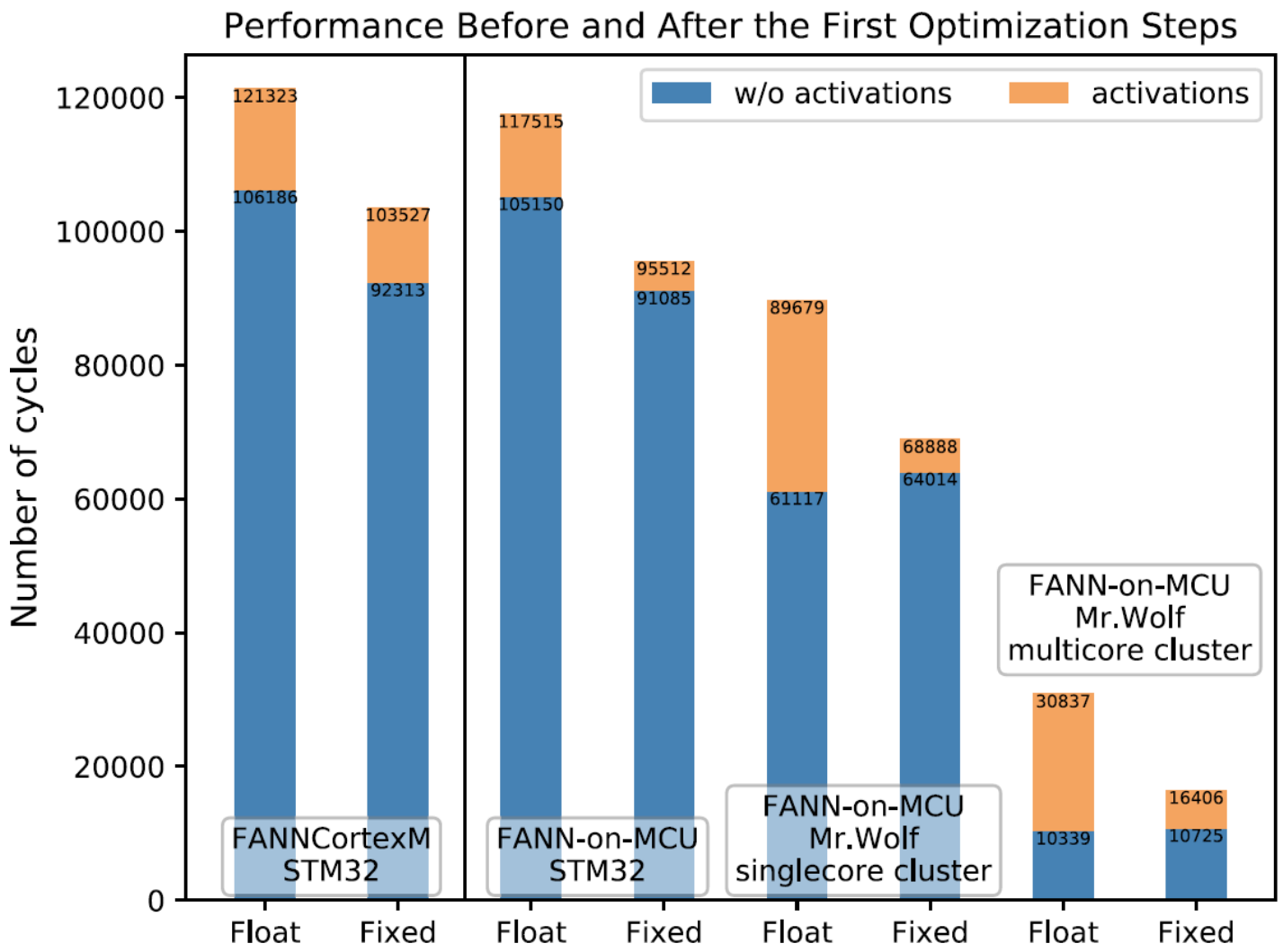
$$N_{\text{tot}} = \sum_{l=1}^{L} (l \ \mathbf{mod} \ 2 + l \ \mathbf{div} \ 2) \cdot d$$

Resultados Experimentais



- Cortex-M4:
  - Melhoria de 3.1% no floating-point e 7.7% no fixed-point
  - fixed-point ≈ 15% mais rápida (7/8 > número de instruções)
  - matrizes de pesos = 88% do tempo total de computação (melhor caso)

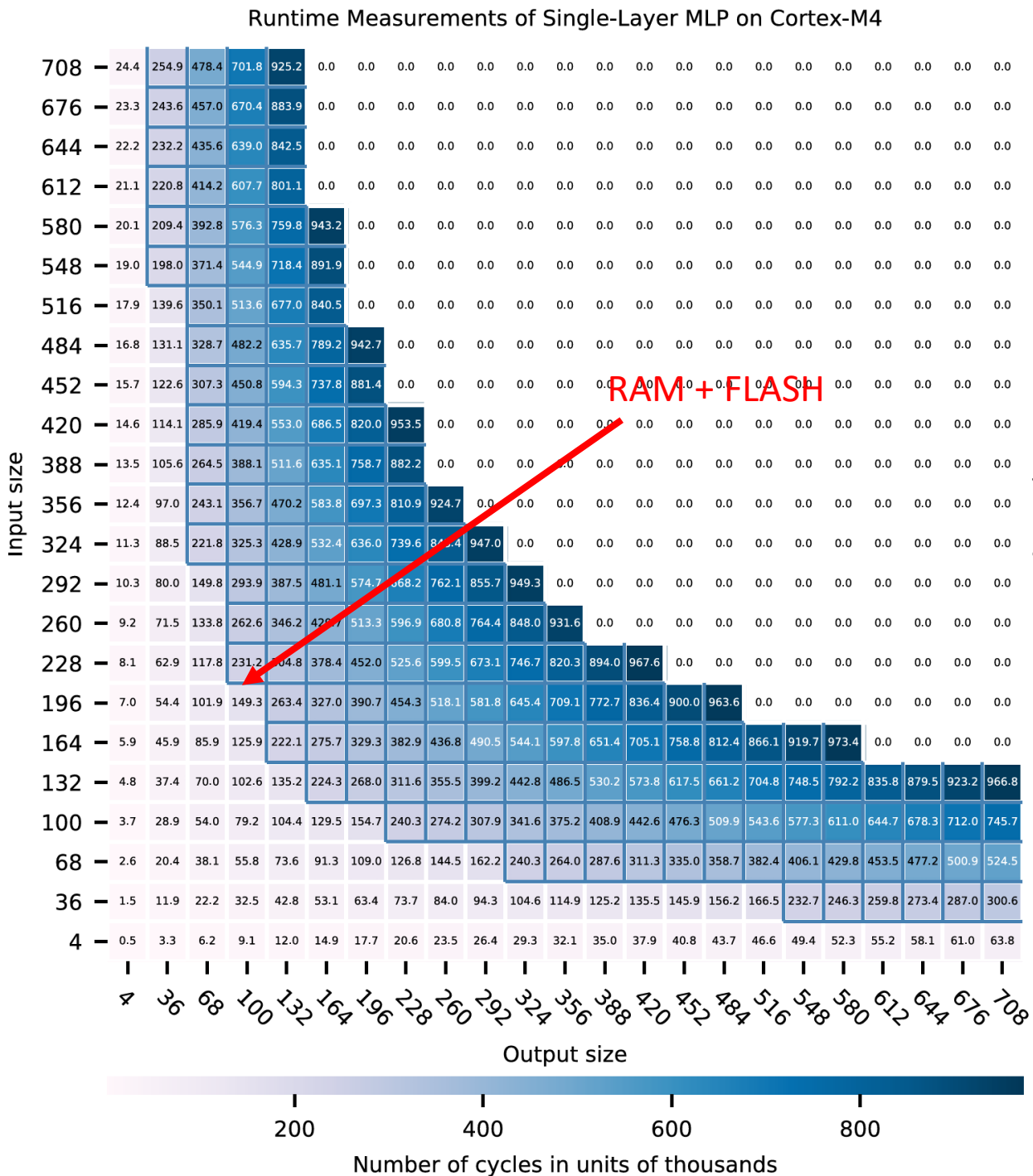
Resultados Experimentais

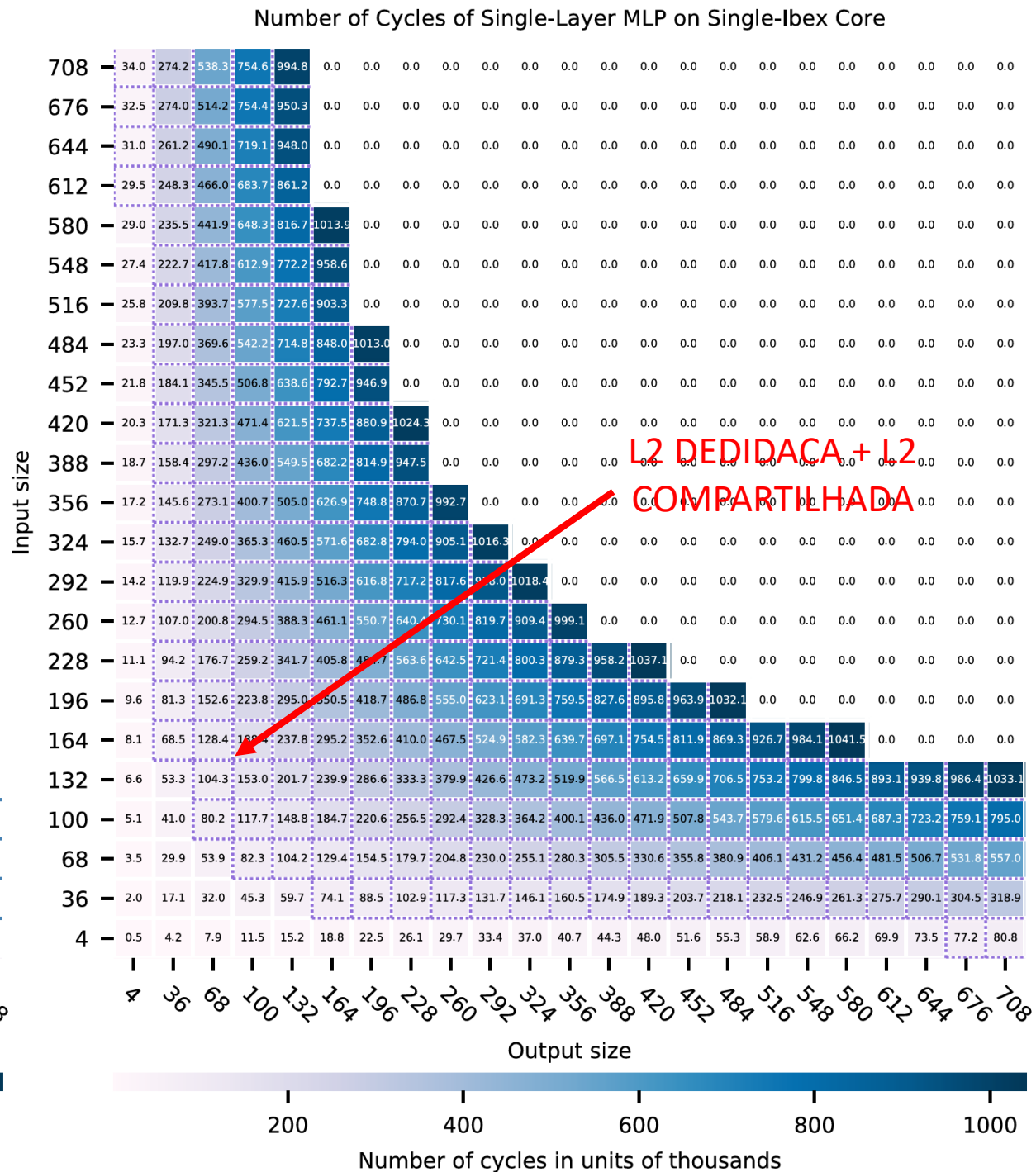


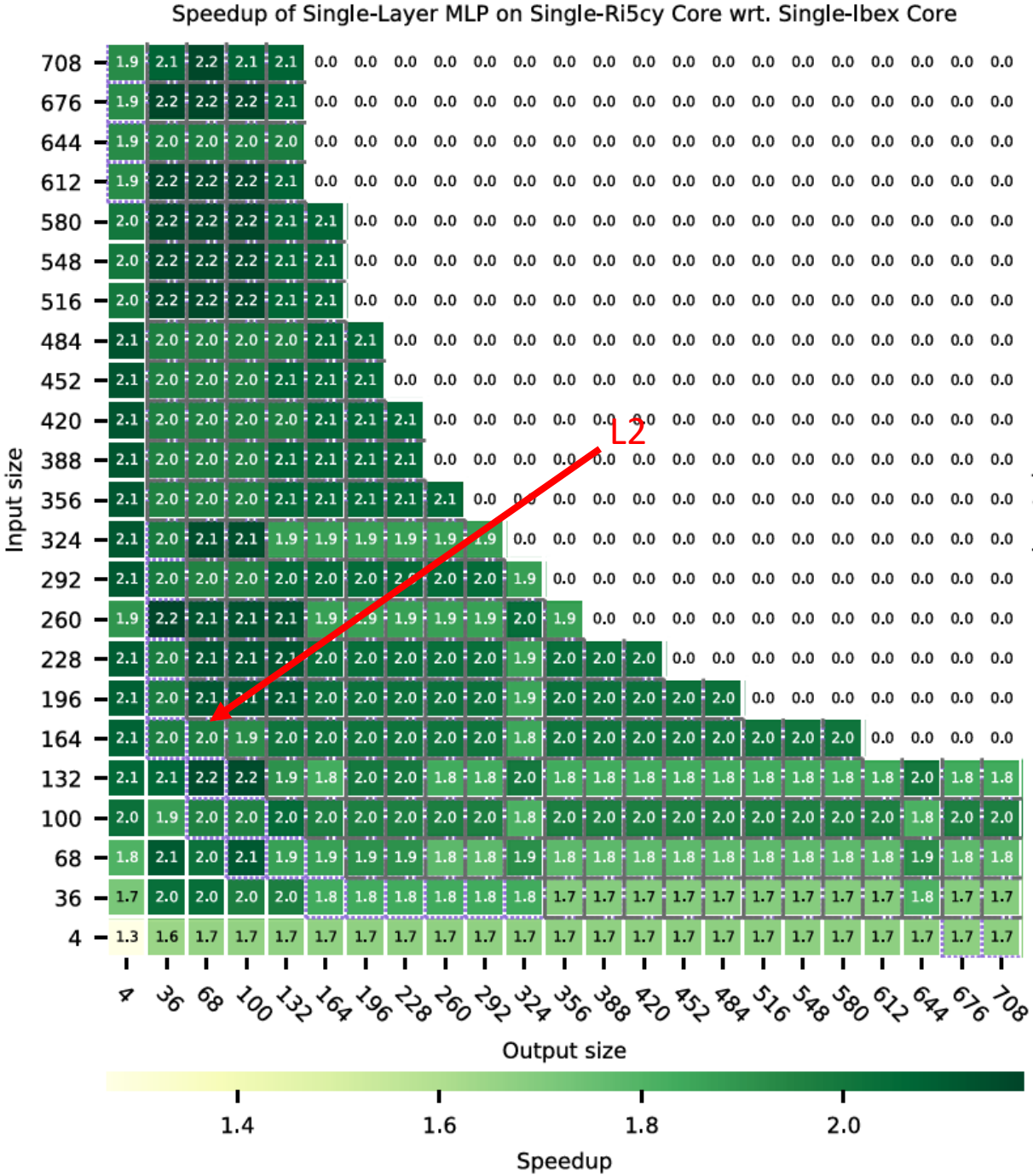
- RI5CY (single core) versus Cortex-M4:
  - 1.3× mais rápido (float)
  - 1.4× mais rápido (fixed)
- Paralelização = até 6× de speedup
- Ativação passa a representar uma fração maior do runtime

Resultados Experimentais

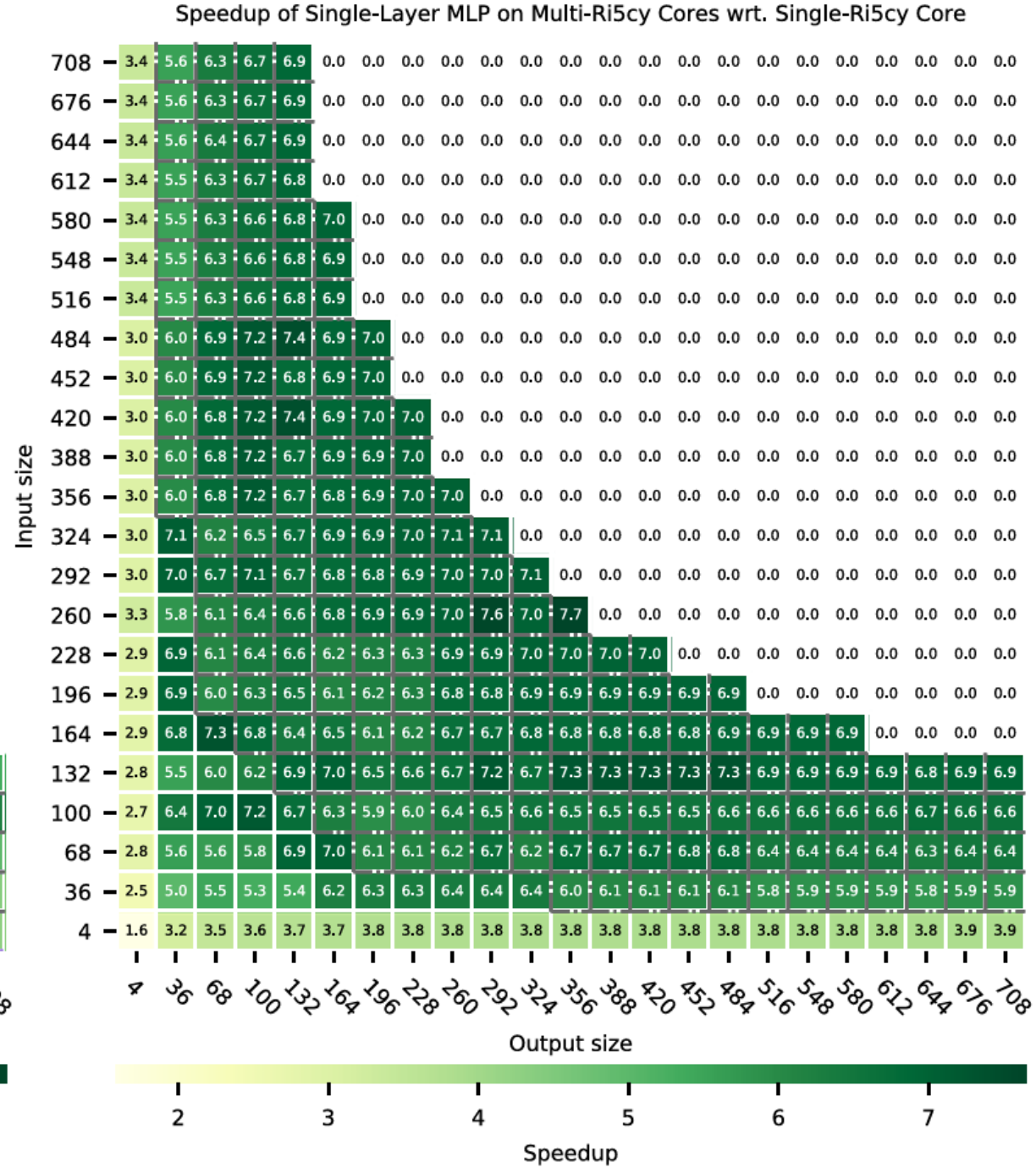
- Análise de desempenho de uma única camada
- Cálculo das matrizes de pesos sem considerar as funções de ativação
- STM32L475VG com ARM Cortex-M4, utilizando implementação em ponto fixo
- Núcleo IBEX utilizando o conjunto básico de instruções RV32IM
- Tempo de execução em número de ciclos de uma única camada ao variar o número de entradas e o número de saídas da camada


Runtime Measurements of Single-Layer MLP on Cortex-M4
RAM + FLASH
Input size
Output size
Number of cycles in units of thousands
200
400
600
800


Number of Cycles of Single-Layer MLP on Single-Ibex Core
L2 DEDIDACA + L2 COMPARTILHADA
Input size
Output size
Number of cycles in units of thousands
200
400
600
800
1000


Speedup of Single-Layer MLP on Single-Ri5cy Core wrt. Single-Ibex Core
Input size
Output size
Speedup
L2
1.4
1.6
1.8
2.0

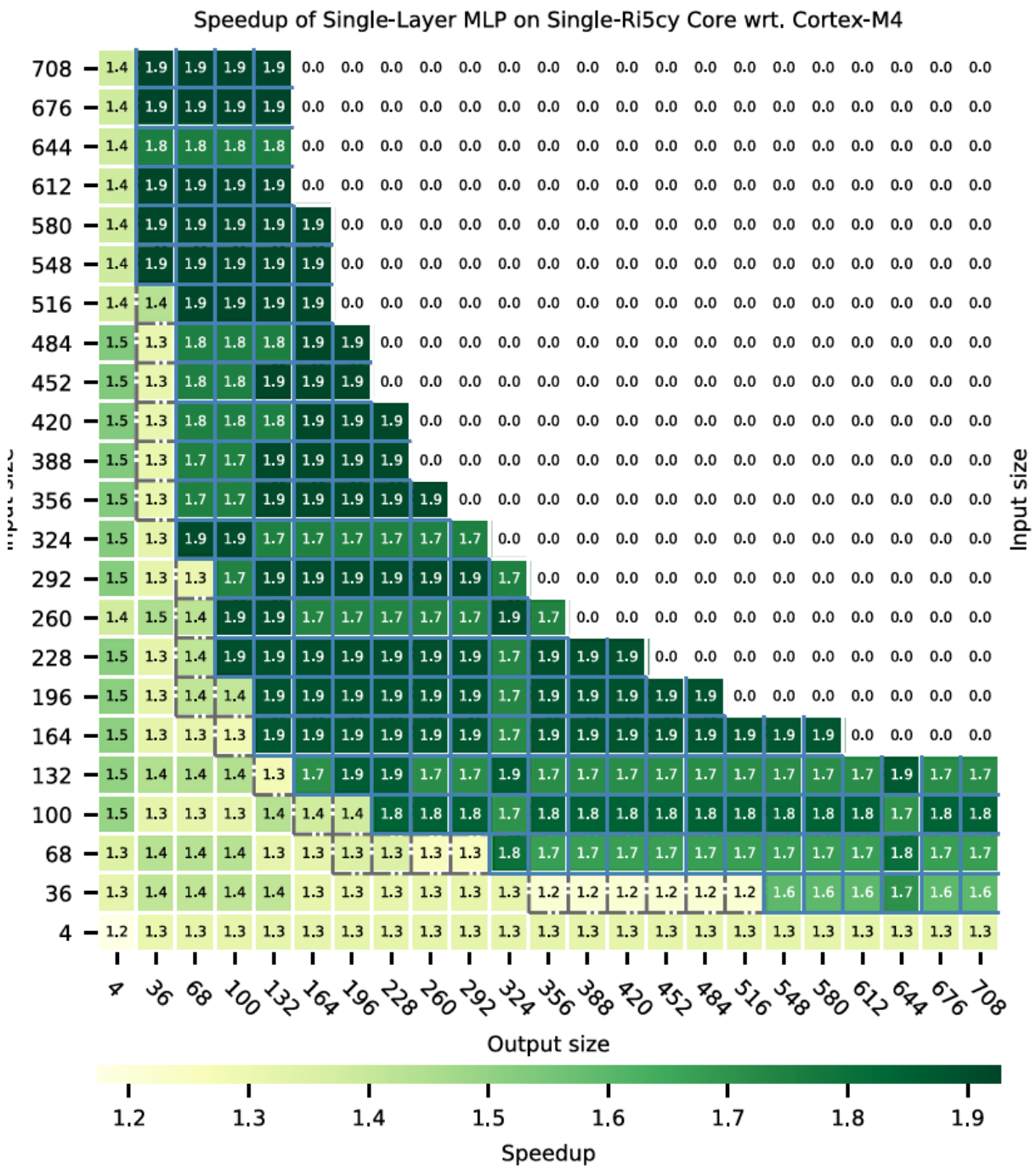
(a)


Speedup of Single-Layer MLP on Multi-Ri5cy Cores wrt. Single-Ri5cy Core
Input size
Output size
Speedup
2
3
4
5
6
7

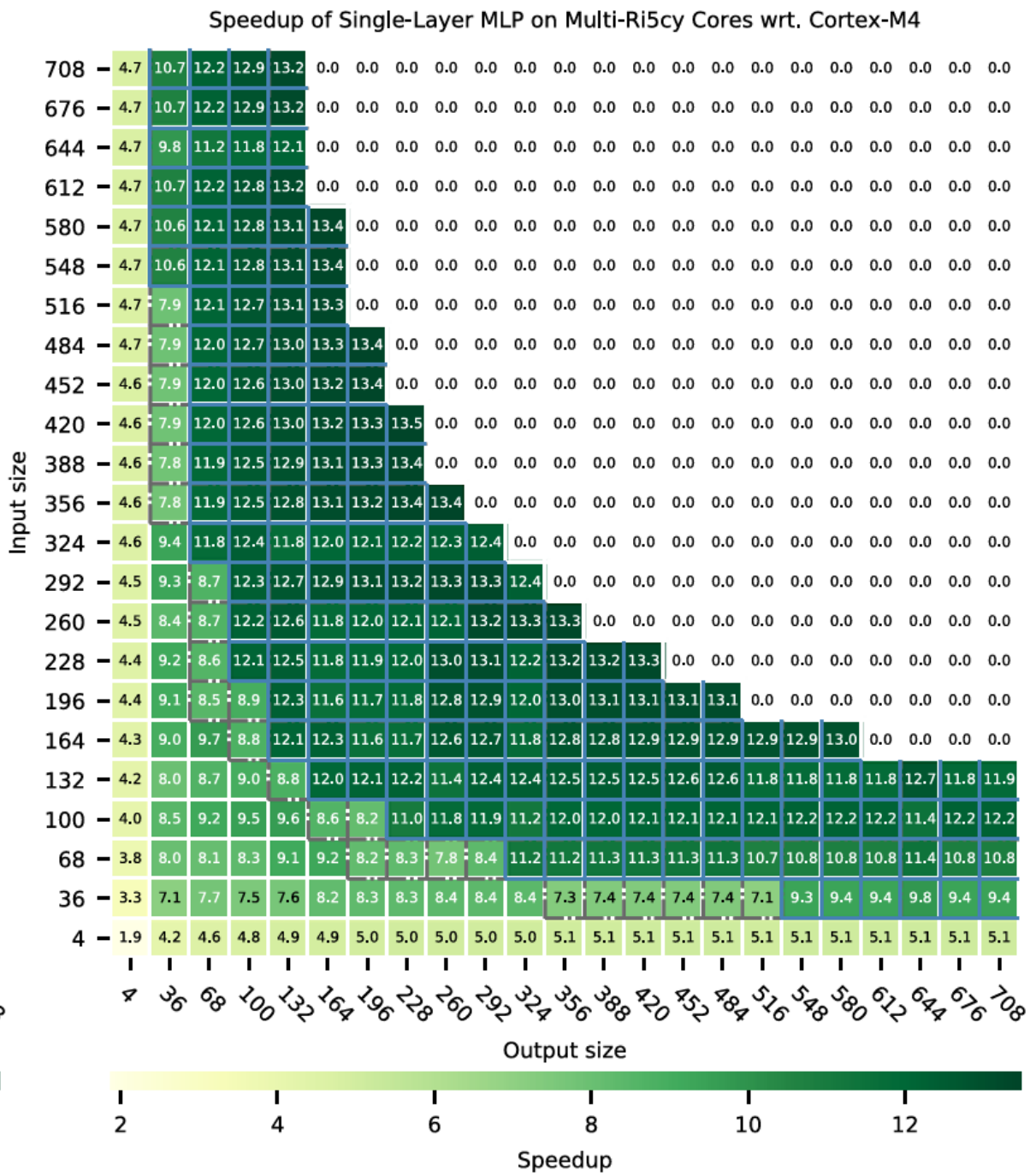
(b)

Resultados Experimentais

- 2,2× de aceleração utilizando um único núcleo RI5CY em relação ao IBEX
- Adição de extensões personalizadas ao conjunto de instruções
- Double-buffering + DMA
- Entradas maiores → maiores ganhos
  - Overhead DMA desprezível frente ao tempo de computação
- Execução paralela em relação à execução com um único núcleo RI5CY → 7,7x de aceleração


Speedup of Single-Layer MLP on Single-Ri5cy Core wrt. Cortex-M4
Input size
Output size
Speedup
1.2
1.3
1.4
1.5
1.6
1.7
1.8
1.9

(a)


Speedup of Single-Layer MLP on Multi-Ri5cy Cores wrt. Cortex-M4
Input size
Output size
Speedup
2
4
6
8
10
12

(b)

Resultados Experimentais

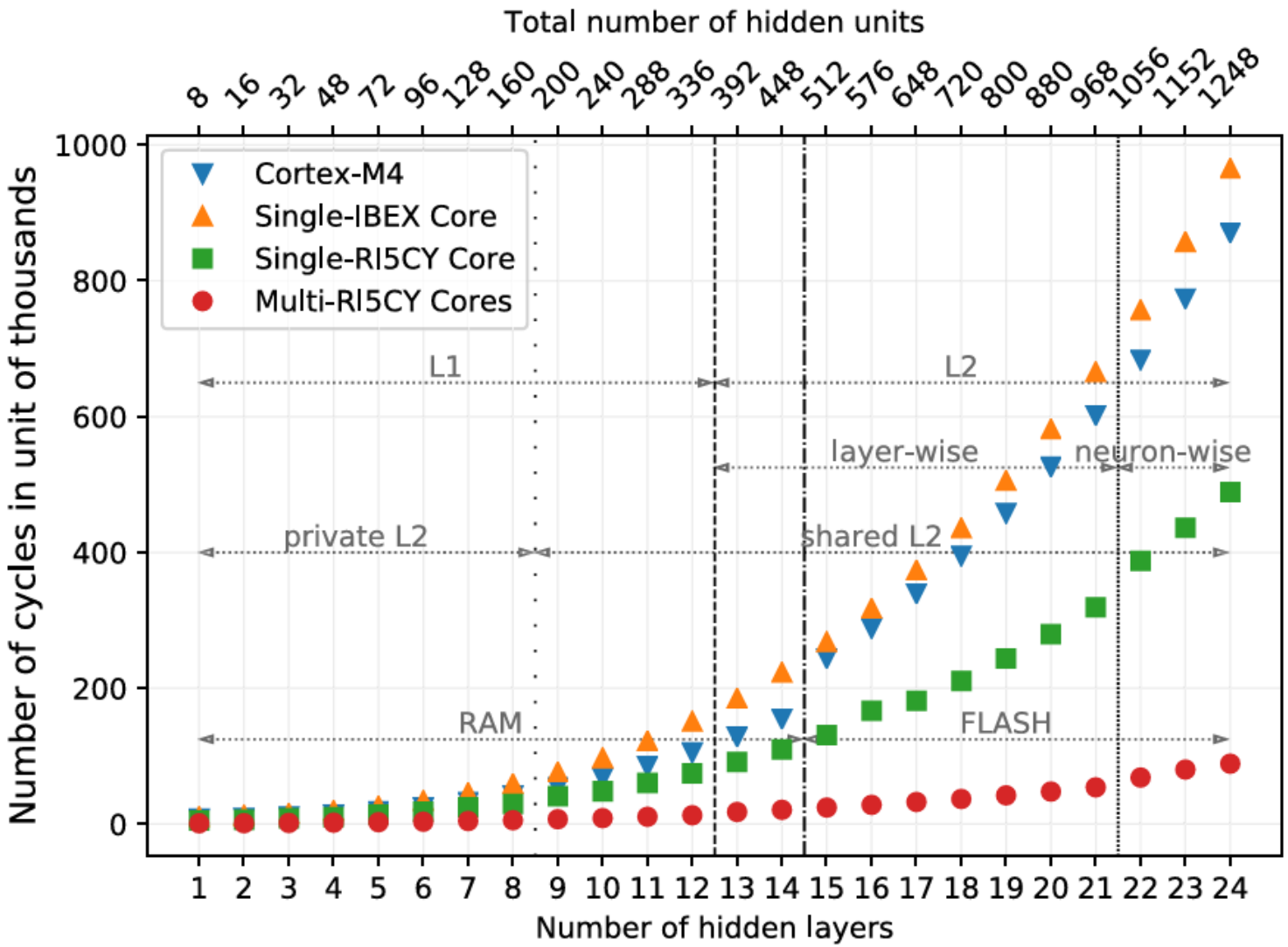
- Single- RI5CY x Cortex-M4 = 2x de aceleração

- Multi RI5CY x Cortex-M4 = 13,5x de aceleração

Resultados Experimentais

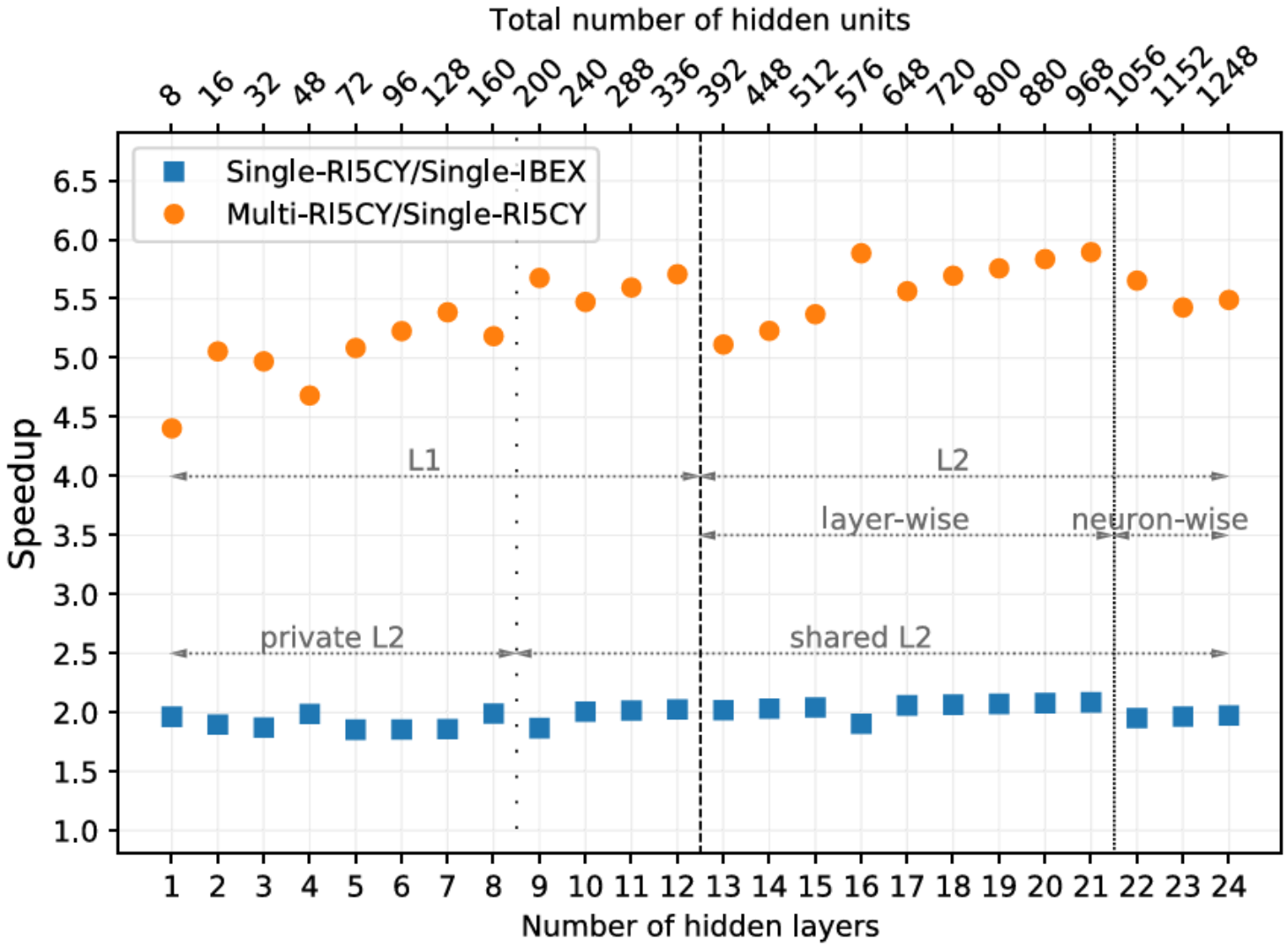
- Desempenho de redes completas
  - Entrada e saída fixa
  - Número de camadas ocultas variável
- Definição do “d”
  - Testar pelo menos dois pontos de medição em cada região de memória
  - Layer-wise e Neuron-wise
  - Maior eficiência energética possível com paralelização

Resultados Experimentais



- Redes pequenas = pouco ganho
- Aumento perceptível com o uso de mais de uma região de memória
  - Pouco perceptível com a paralelização
- Aumento perceptível na região neuron-wise

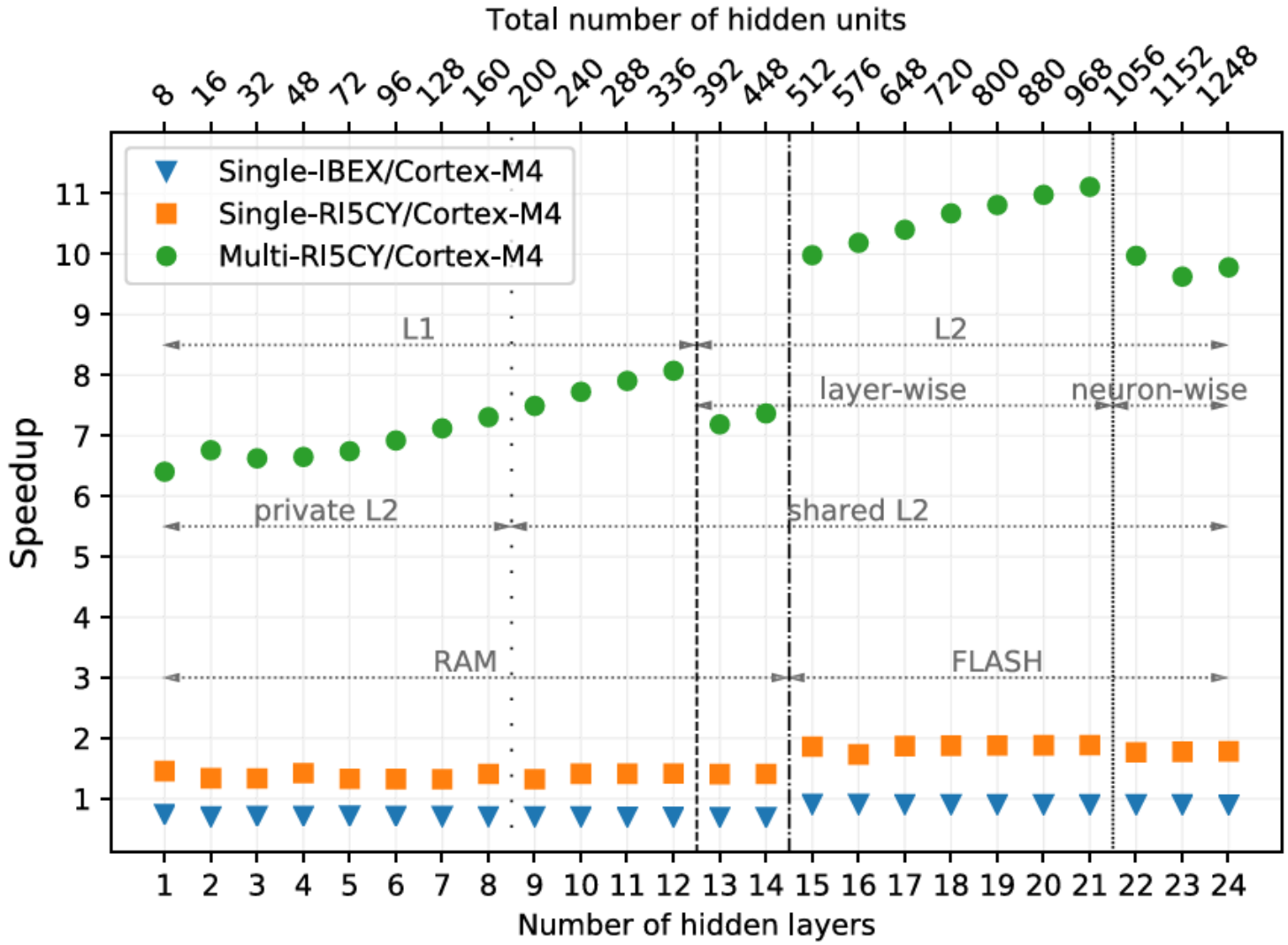
Resultados Experimentais



(a)

- Speedup obtido com paralelização é menor em redes pequenas
  - Overhead da paralelização
- Ligeira perca de Speedup com a troca de região de memória

Resultados Experimentais



(b)

- IBEX ligeiramente mais lento que M4. Porém o desempenho se assemelha em quando M4 necessita da Flash
- Um único núcleo RI5CY é quase duas vezes mais rápido que um núcleo Cortex-M4
  - Extensões customizadas do conjunto de instruções (ISA)
- O speedup paralelo aumenta de forma constante com o crescimento do tamanho da rede

Resultados Experimentais

- Reconhecimento de Gestos da Mão - Aplicação A
  - EMG de superfície e IMU de 9 eixos
  - Objetivo: reconhecer 10 gestos da mão
  - 76 características no domínio do tempo → entrada para uma MLP
  - 3 camadas ocultas → 300, 200 e 100 neurônios
  - 10 classes de saída
  - maior acurácia de classificação obtida foi de 85,58%.

Resultados Experimentais

- Detecção de Quedas em Idosos - Aplicação B
  - Sensores de pressão nas palmilhas dos calçados
  - Acelerômetros: cabeça, pelve e pernas
  - 117 neurônios de entrada
  - 20 neurônios na camada oculta
  - 2 saídas
  - Acurácia obtida foi de 84%

Resultados Experimentais

- Classificação de Atividades Humanas - Aplicação C
  - Acelerômetro triaxial posicionado na cintura.
  - Cinco classes diferentes de saída
  - 7 características de entrada
  - 1 camada oculta com 6 neurônios
  - Acurácia obtida foi de 94,6%

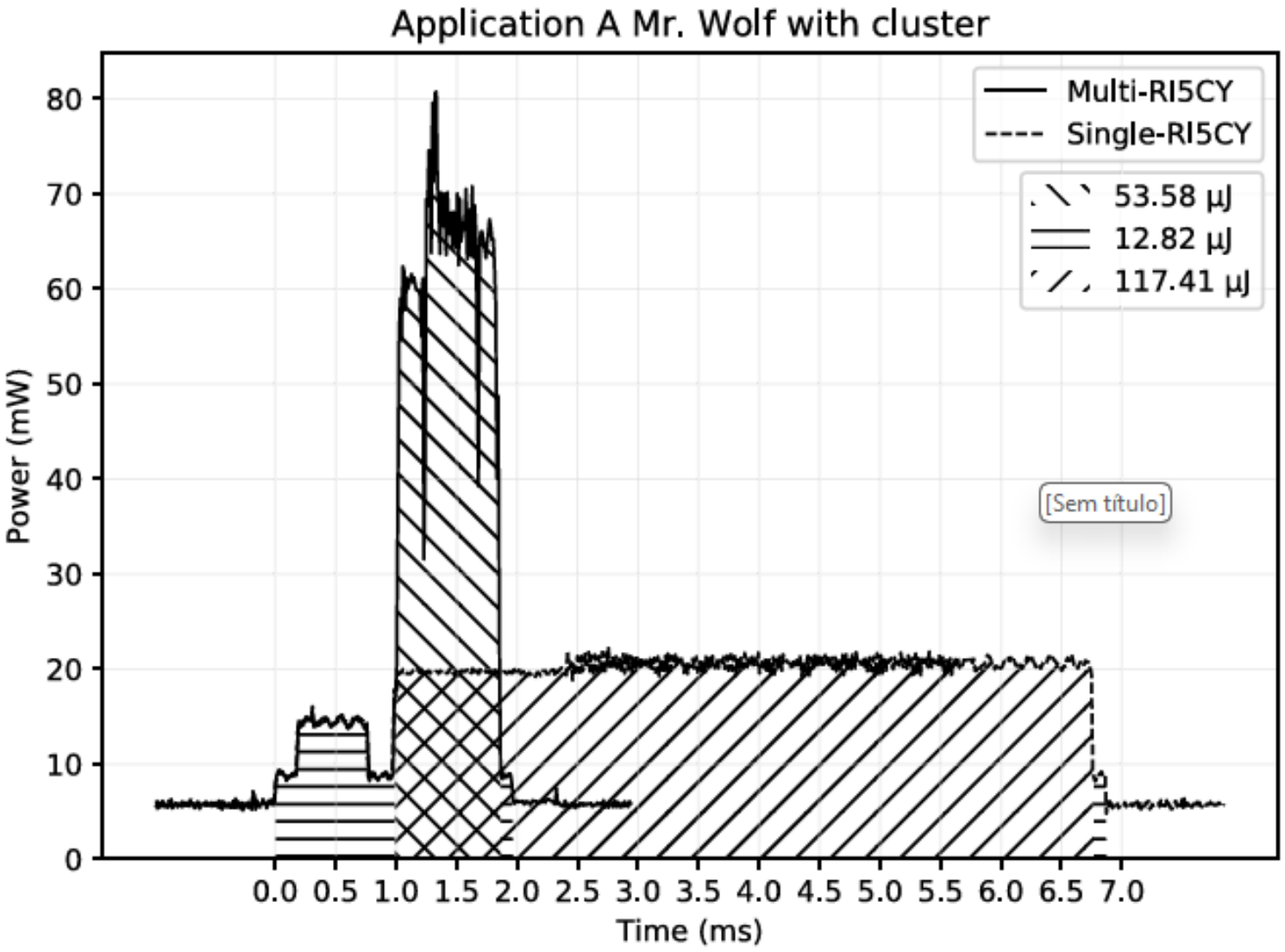
Resultados Experimentais

- InfiniWolf
  - Nordic nRF52832 com ARM Cortex-M4
  - PULP Mr. Wolf
- nRF52832 → 64 MHz
- Mr. Wolf → 100 MHz
- analisador de potência Keysight N6705C → 0,1024 ms

Resultados Experimentais

- Desempenho end-to-end
  - Ativar o cluster
  - Copiar dados para a memória
  - Executar a rede neural
  - Desligar o cluster
- Overhead → 13 µJ

Resultados Experimentais



Resultados Experimentais

TABLE II
MEASURED RUNTIME AND AVERAGE POWER CONSUMPTION DURING COMPUTATION ON APPLICATION SHOWCASES. RELATIVE IMPROVEMENTS PER INFERENCE WITH RESPECT TO ARM CORTEX-M4 IMPLEMENTATION IN EXECUTION TIME SPEEDUP AND IN ENERGY REDUCTION ARE REPORTED IN PARENTHESIS

App.	nRF52832	PULP Mr. Wolf		
	ARM Cortex-M4	IBEX	Single-RI5CY	Multi-RI5CY
A	17.6 ms	11.4 ms (1.54×)	5.7 ms*(3.09×)	0.8 ms*(22×)
	10.44 mW	10.75 mW	20.35 mW	61.79 mW
	183.74 µJ	122.55 µJ (-33.30%)	116.00 µJ (-36.87%)	49.43 µJ (-73.10%)
B	0.4 ms	0.3 ms (1.33×)	0.14 ms*, †(2.86×)	0.03 ms*, †(13.33×)
	11.21 mW	9.52 mW	17.54 mW	22.18 mW
	4.48 µJ	2.86 µJ (-36.16%)	2.46 µJ (-45.09%)	0.6654 µJ (-85.15%)
C	0.03 ms†	0.02 ms†(1.5×)	0.01 ms*, †(3×)	0.004 ms*, †(7.5×)
	9.74 mW	7.31 mW	16.91 mW	17.17 mW
	0.2922 µJ	0.1462 µJ (-49.97%)	0.1691 µJ (-42.13%)	0.06868 µJ (-76.50%)

* In addition around 1~1.3 ms for cluster activation, initialization, and deactivation with an average power consumption of 11.88 mW.
† The runtime is below or very close to the precision of the measuring instrument, hence it is calculated from the measured number of cycles.

Resultados Experimentais

- Redes neurais muito pequenas podem não se beneficiar do paralelismo
- Para aplicações com classificação contínua (típicas de IoT), a execução paralela torna-se claramente vantajosa
- A arquitetura paralela baseada em cluster torna-se energeticamente mais eficiente após aproximadamente seis classificações → Aplicação B

$$13 + 0{,}67 \times N < 2{,}86 \times N \rightarrow N = 6$$

Análise crítica do trabalho

- Pontos fortes:
  - Avaliação experimental robusta comparando diversos cenários e condições
  - Métodos claros e conclusivos
- Limitações
  - Hardware de testes
  - Trabalho altamente correlacionado as características das plataformas testadas

Análise crítica do trabalho

- Possíveis melhorias
  - Ensaio de consumo
- Aplicações práticas
  - Sistemas embarcados em geral com processamento em borda

Conclusão

- Viabilidade de implementar IA em microcontroladores
- Importância da otimização de hardware e paralelização
  - 7,1× de aceleração em relação a uma implementação em um único núcleo
  - até 13,5× mais desempenho em comparação com um ARM Cortex-M4.
- Importância da analise entre paralelização e consumo

Conclusão

- Trabalhos futuros:
  - Realizar experimentos em outros microcontroladores além de ARM Cortex-M e da arquitetura PULP RISC-V architecture, incluindo novos processadores baseados em RISC-V e plataformas com aceleradores de IA
  - Investigar o impacto de diferentes quantidades de núcleos ativos e estratégias de paralelização, analisando o equilíbrio entre desempenho, consumo de energia e latência

Perguntas?

Referências

- X. Wang, M. Magno, L. Cavigelli and L. Benini, "FANN-on-MCU: An Open-Source Toolkit for Energy-Efficient Neural Network Inference at the Edge of the Internet of Things," in IEEE Internet of Things Journal, vol. 7, no. 5, pp. 4403-4417, May 2020, doi: 10.1109/JIOT.2020.2976702.

- STMicroelectronics. STM32Cube.AI. Disponível em: https://stm32ai.st.com/stm32-cube-ai/. Acesso em: 16 mar. 2026.

- ARM. CMSIS – Common Microcontroller Software Interface Standard. Disponível em: https://www.arm.com/technologies/cmsis. Acesso em: 16 mar. 2026.

- FANN-on-MCU. GitHub Disponível em: https://github.com/pulp-platform/fann-on-mcu. Acesso em: 16 mar. 2026.

Obrigado!

https://forms.gle/hNAmq6qdL2Z24e299